

Fundamentos en AWS: Servicios Clave

Lambda, S3, DynamoDB, Aurora, Athena, QuickSight y Terraform

Santiago García

Solutions Architect, AWS

Agenda

- 1 **Introducción a AWS y la nube**
- 2 Serverless y AWS Lambda
- 3 Almacenamiento de objetos: Amazon S3
- 4 Base de datos NoSQL: Amazon DynamoDB
- 5 Base de datos relacional: Amazon Aurora
- 6 Analítica serverless: Amazon Athena
- 7 Business Intelligence: Amazon QuickSight
- 8 Infraestructura como Código: Terraform
- 9 **DEMOS en vivo + preguntas**



Introducción a *AWS* y la nube



¿Qué es la computación en la nube?



Entrega bajo demanda de recursos de TI a través de internet con un modelo de **pago por uso**.

¿Cómo se benefician los clientes de la nube?

Agilidad

Los equipos pueden experimentar e innovar rápida y frecuentemente

Innovación

Enfócate en los diferenciadores del negocio, no en la infraestructura

Ahorro de costos

Paga solo por lo que usas, aprovechando economías de escala

Elasticidad

Escala hacia arriba o hacia abajo fácilmente según las necesidades del negocio

Seguridad

Obtén el control que necesitas para operar tu negocio de forma segura

Alcance global

Infraestructura global en la nube extensa, confiable y segura

Casos de éxito de clientes

AWS es aliado estratégico de clientes en sector Financiero

AWS está diseñada para ser la nube más segura y los 120+ servicios de AWS que almacenan datos ofrecen la capacidad de cifrarlos.



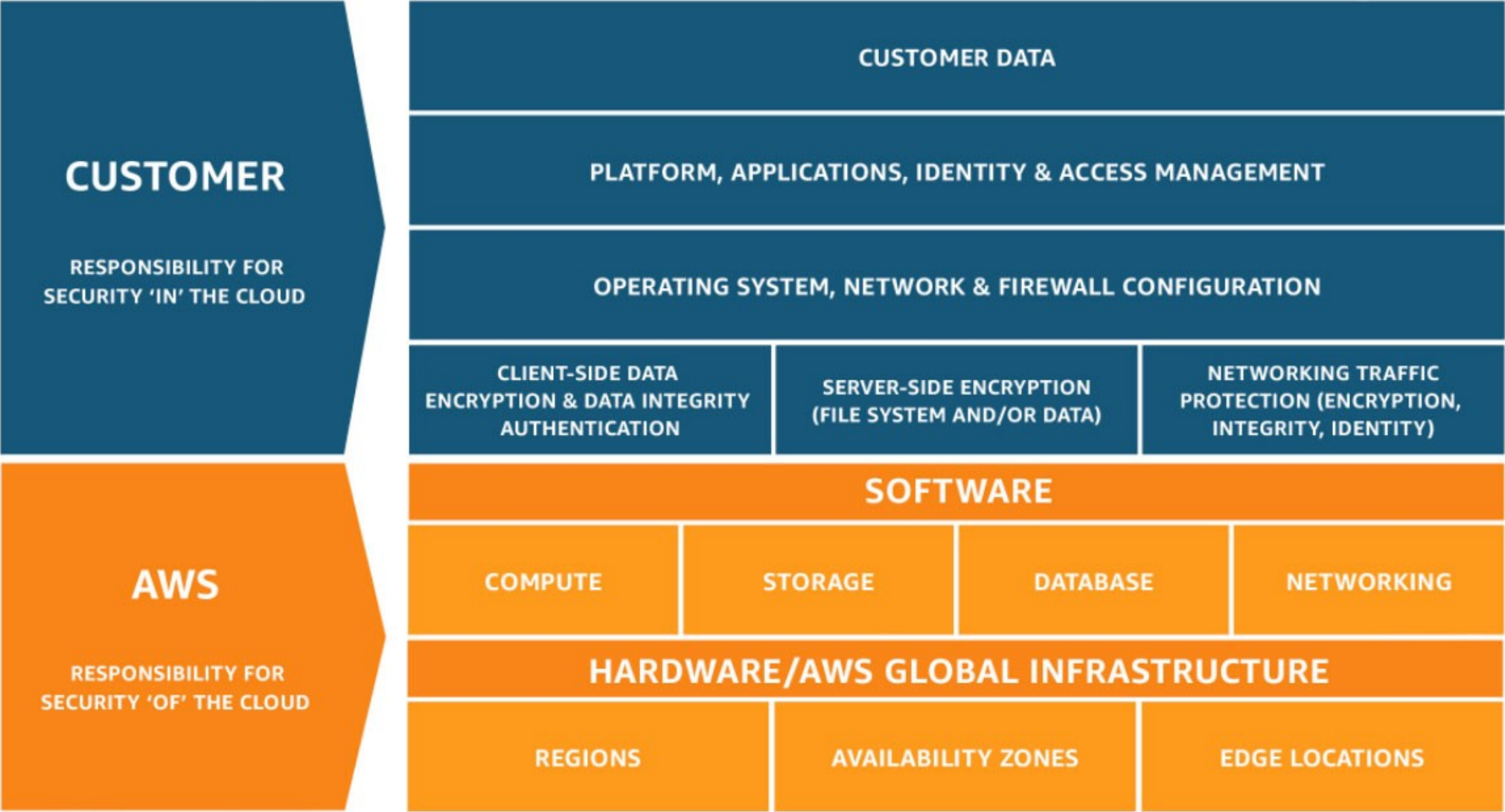
¿SABÍAS QUE?

AWS ofrece:

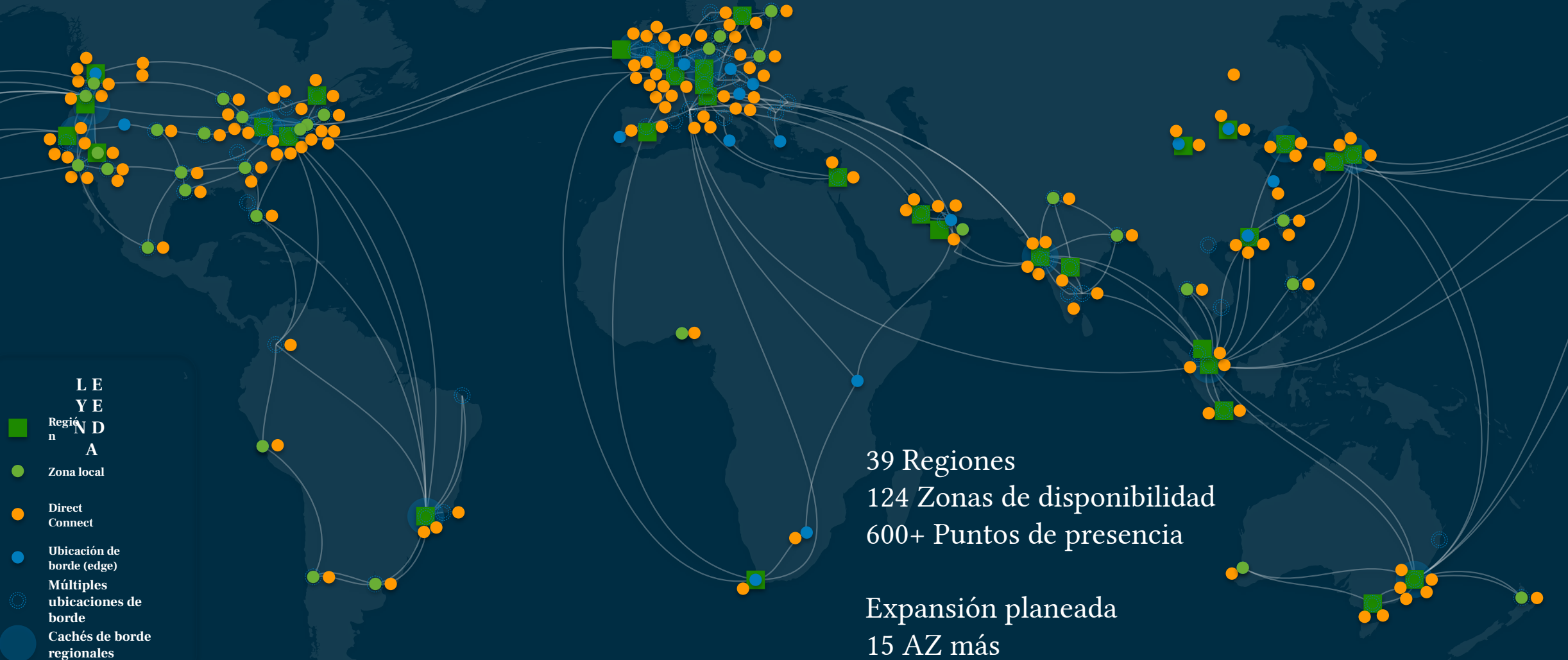
300+

Servicios y funcionalidades de seguridad, cumplimiento y gobierno.

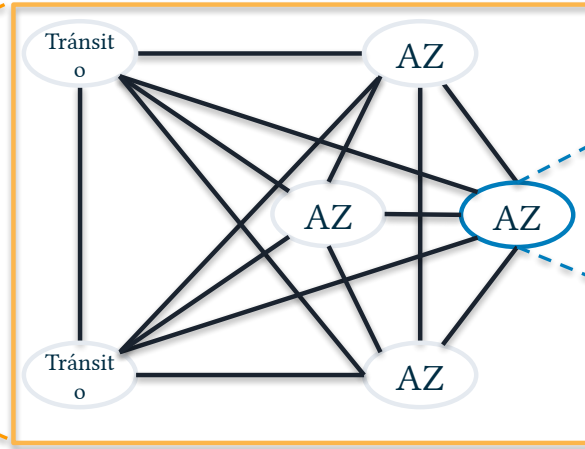
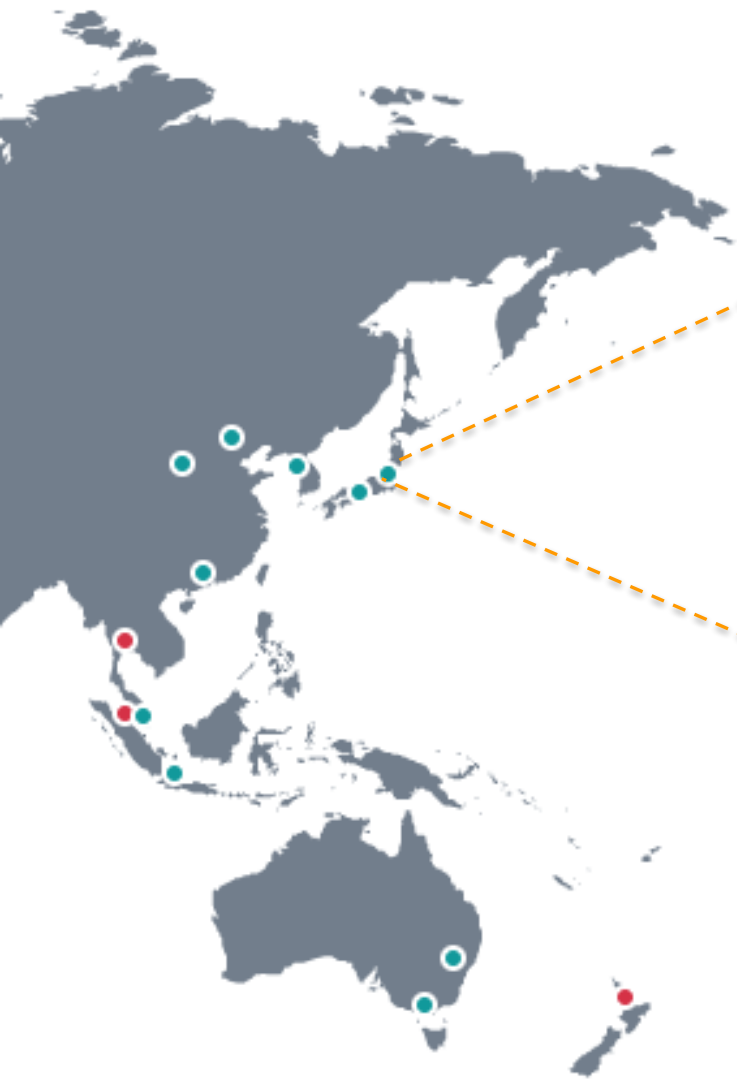
Modelo de responsabilidad compartida



Infraestructura global de AWS

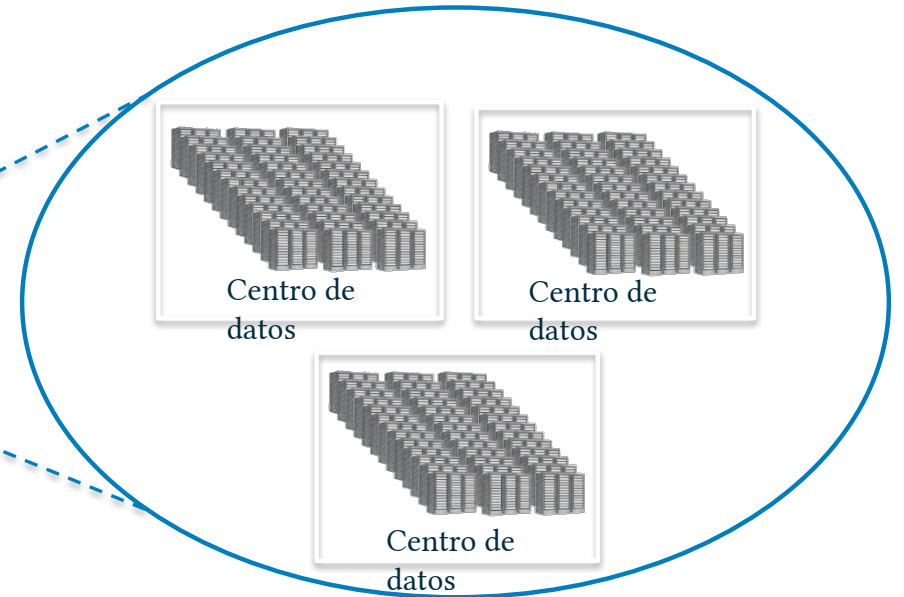


Diseño de una Región de AWS



AWS Region

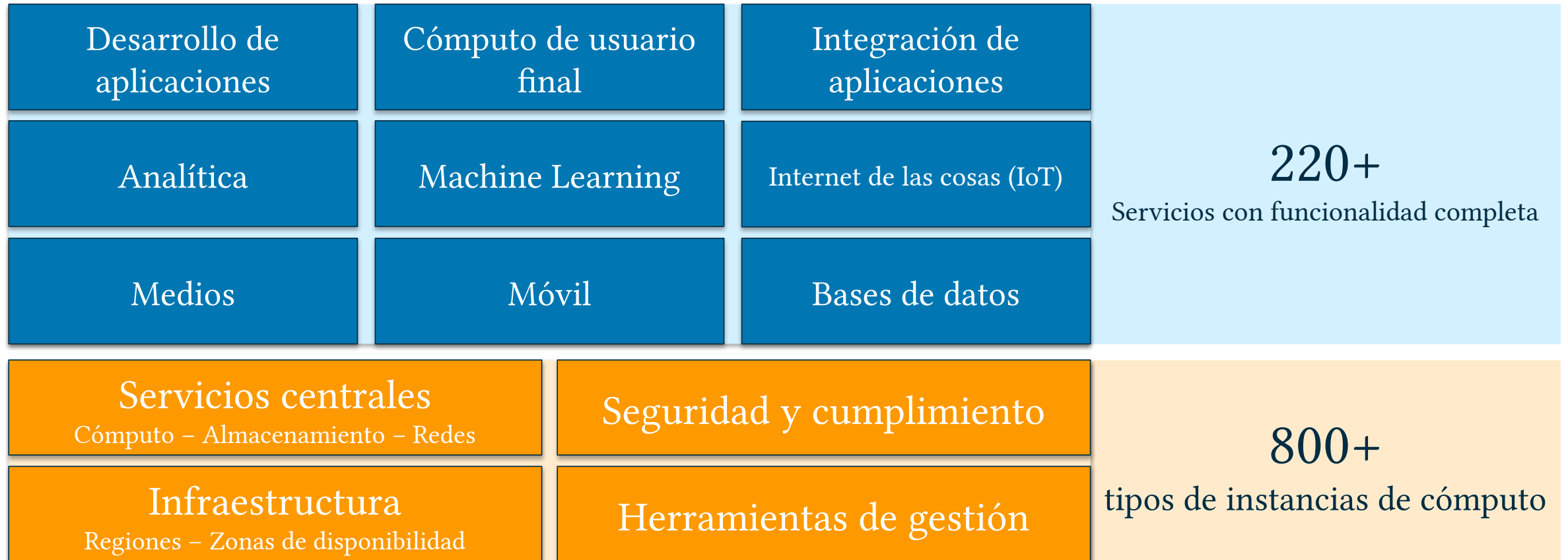
Una Región es una ubicación física en el mundo donde agrupamos centros de datos. A cada grupo de centros de datos lógicos lo llamamos **Zona de disponibilidad**.



Zona de disponibilidad (AZ) de AWS

Las Zonas de disponibilidad están formadas por uno o más centros de datos independientes, cada uno con energía, red y conectividad redundantes, alojados en instalaciones separadas.

Soporta prácticamente cualquier carga de trabajo en la nube





Cómputo serverless: *AWS* Lambda

Repasemos el concepto de “Serverless”

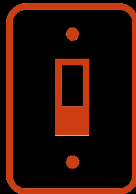
Aprovecha al MÁXIMO las bondades de la nube para modernizar/integrar y acelerar la innovación.



NO debes provisionar (ni operar) infraestructura



Escalado automático



Pago-Par-Uso (pay-as-you-go)



Alta Disponibilidad y Resiliencia

AWS Serverless (servicios referentes)

TIP: La mayoría de servicios de integración, son de tipo Serverless

Primitivas...

(Ejemplo: cómputo, buses, mensajes...)



AWS
Lambda



AWS
Fargate



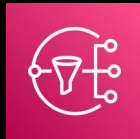
Amazon
EventBridge



Amazon
Bedrock



Amazon
ECS



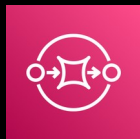
Amazon
SNS



Amazon
S3



Amazon
EFS



Amazon
SQS



Amazon
API
Gateway

Periféricos

(Ejemplos: DBs, Orquestación, Analítica...)



Amazon
Aurora
Serverless



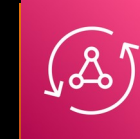
Amazon
DynamoDB



Amazon
Kinesis



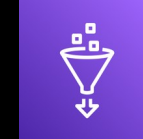
AWS
Step
Functions



AWS
AppSync



Amazon
CloudWatch



AWS Glue



Amazon
OpenSearch



Amazon
QuickSight
(QuickSuite)

¿Qué es AWS Lambda?

EL PODER DE CÓMPUTO DETRÁS DE LAS FUNCIONES SERVERLESS



Ejecutamos tu **código** en
respuesta a eventos o
solicitudes

Sin aprovisionar ni
administrar **servidores**

Escala automáticamente
en respuesta al tráfico

Pago por uso;
por **milisegundo**

Empaqueta como archivo
.zip o **imagen de
contenedor**

Runtimes integrados e
integraciones nativas
con **más de 200** servicios
de AWS y aplicaciones
SaaS

¿Cómo funciona AWS Lambda?



Modos de invocación de Lambda

Síncrono

solicitud/respuesta

-  Application Load Balancer
-  Amazon API Gateway
-  AWS Lambda function URL

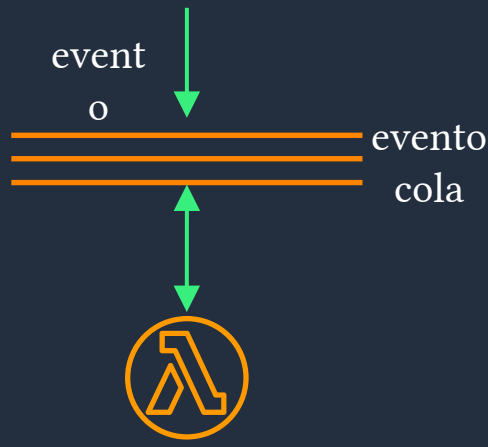


Función Lambda

Asíncrono

evento

-  Amazon EventBridge
-  Amazon S3

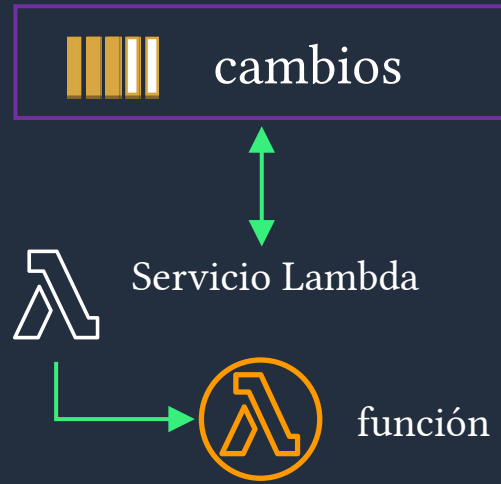


Función Lambda

Stream

basado en polling

-  Amazon DynamoDB
-  Amazon Kinesis



función

Anatomía de una función Lambda

Función handler

- Función que se ejecuta en cada invocación
- Procesa el evento entrante

Evento

- Datos de la invocación enviados a la función
- Su forma depende de la fuente del evento

Contexto

- Información adicional del servicio Lambda
- Ejemplos: ID de la solicitud, tiempo restante

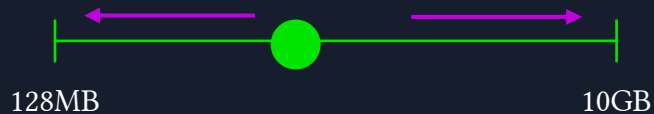
app.py

```
def handler(event, context):  
    msg = 'Hello {}'.format(  
        event['name']  
    )  
    return { 'message': msg }
```

Configuración de una función Lambda

Potencia (memoria)

- Selecciona entre 128 MB y 10 GB
- CPU y red se asignan proporcionalmente
- Ajusta la potencia (power tuning) para equilibrar costo y velocidad



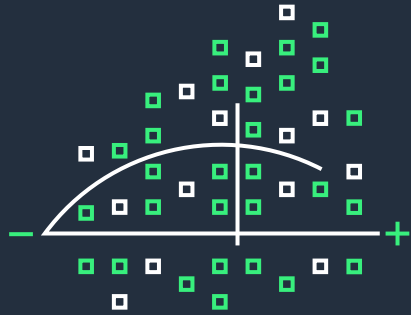
Modelo de permisos

- El rol de ejecución otorga a la función acceso a recursos mediante IAM
- La política basada en recursos controla la invocación desde otras cuentas u organizaciones de AWS y desde servicios de AWS

Principales casos de uso de AWS Lambda



**Automatización
n TI**



**Procesamiento
de datos y
streams**



**Aplicaciones
web**



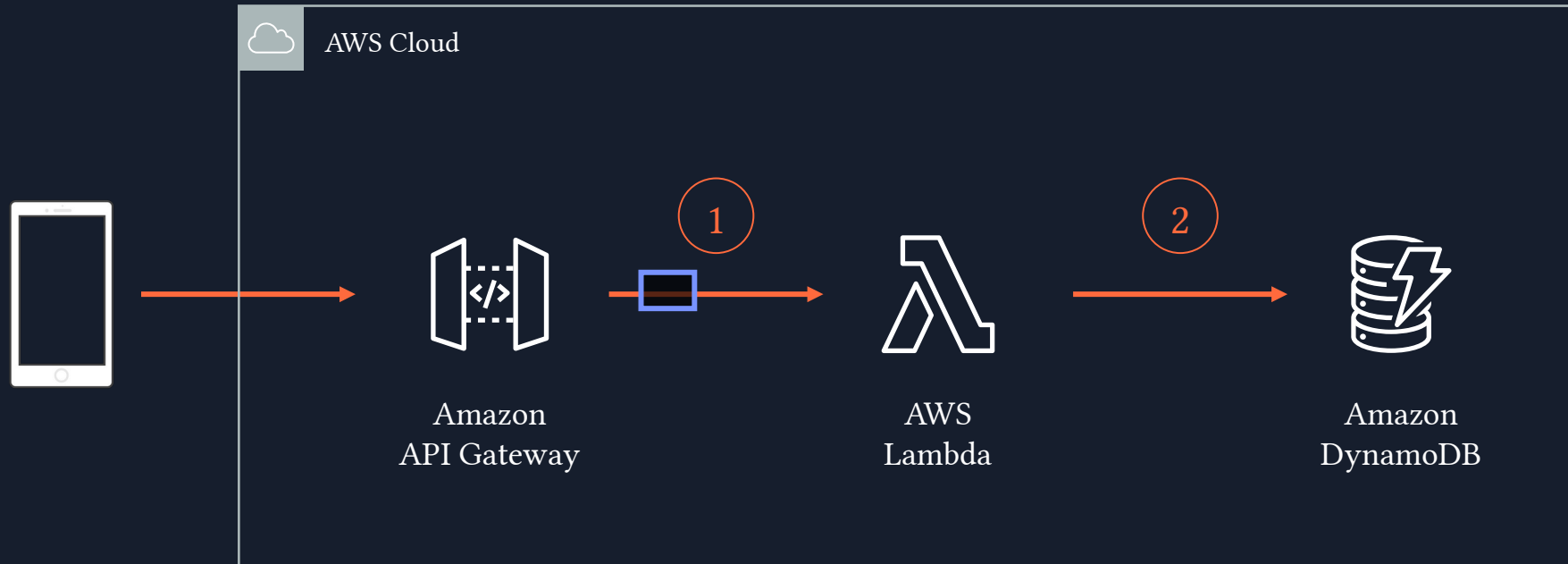
**Aplicaciones
basadas en
eventos**



**Machine
learning**

Ejemplo 1: Lambda para exponer API

HIGHLY-SCALABLE MICROSERVICES



1. API Gateway "traduce" la solicitud HTTP entrante en un evento (payload)

2. Lambda lee y escribe datos en el almacén de datos

Ejemplo 2: Procesamiento de archivos cargados

REDIMENSIONAR FOTOS, EXTRAER TEXTO, TRADUCIR, ETC.



1. Objeto cargado a un bucket de Amazon S3
2. **Invocación asíncrona** de la función Lambda; el evento incluye:
 - Nombre del bucket
 - Clave (key) del objeto

Buenas prácticas: AWS Lambda

Buenas prácticas

1 Funciones pequeñas

Una responsabilidad por función y handler delgado. La lógica de negocio va en módulos aparte para poder probarla sin desplegar.

2 Reutiliza el entorno

Inicializa clientes del SDK, conexiones y configuración fuera del handler: se reutilizan en invocaciones calientes y reducen latencia.

3 Memoria y timeout

Ajusta memoria con Lambda Power Tuning (más memoria = más CPU). Timeouts realistas; recuerda que API Gateway corta a ~30 s.

4 Permisos y secretos

Un rol IAM de mínimo privilegio por función. Secretos en Secrets Manager o Parameter Store, nunca en código ni variables en claro.

5 Piensa asíncrono

Usa SQS/EventBridge cuando no necesites respuesta inmediata. Configura DLQ, reintentos idempotentes y concurrencia reservada para proteger downstream.

6 Observabilidad y despliegue

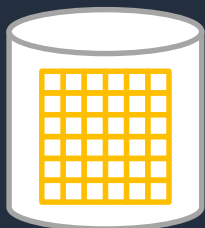
Logs estructurados (JSON), métricas y trazas con X-Ray / Powertools. Despliega con IaC, versiones y alias para rollbacks seguros.



Almacenamiento de objetos: Amazon S3



Bloque vs. Archivo vs. Objeto



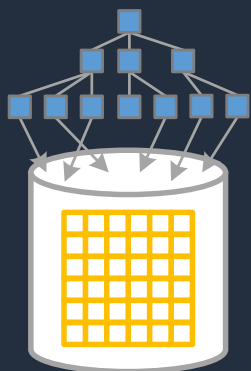
Almacenamiento en bloque

Almacenamiento en bruto

Datos organizados como un arreglo de bloques no relacionados

El sistema de archivos del host ubica los datos en disco

Ej.: discos duros, arreglos de almacenamiento SAN (Storage Area Network)

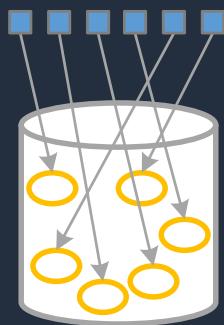


Almacenamiento de archivos

Bloques de datos no relacionados administrados por un sistema de archivos

El sistema de archivos nativo ubica los datos en disco

Ej.: appliances NAS (Network Attached Storage), servidores de archivos Windows, NetApp ONTAP



Almacenamiento de objetos

Almacena contenedores virtuales que encapsulan los datos, sus atributos, metadatos e IDs de objeto

Acceso a los datos por API

Basado en metadatos, en políticas, etc.

Ej.: Amazon S3, Ceph, OpenStack Swift

AWS ofrece el portafolio de almacenamiento más amplio de la industria

OBJETO



Amazon S3

BLOQUE



Amazon EBS

ARCHIVO



Amazon EFS



Amazon FSx for NetApp ONTAP



Amazon FSx for Windows File Server



Amazon FSx for Lustre



Amazon FSx for OpenZFS



Amazon File Cache

BACKUP



AWS Backup

TRANSFERENCIA Y MIGRACIÓN DE DATOS



AWS Storage Gateway



AWS DataSync



AWS Transfer Family



AWS Snowball



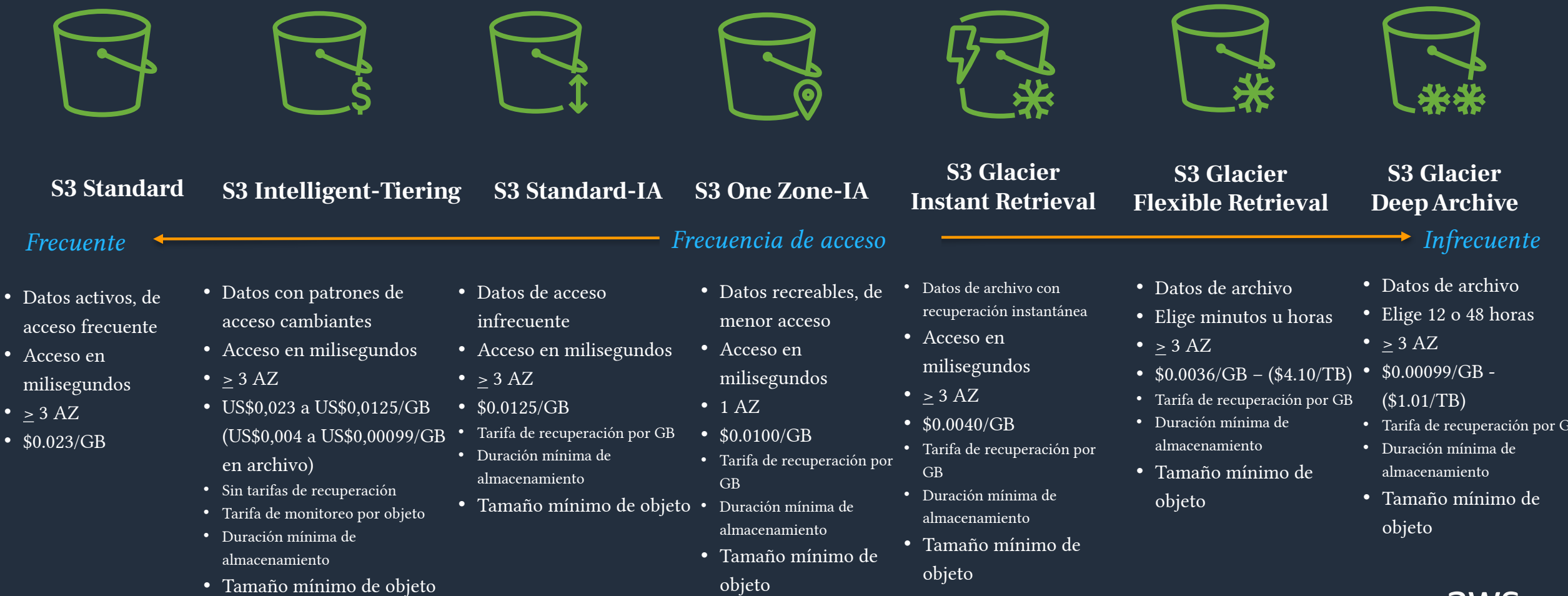
AWS Snowcone

Amazon S3 (Simple Storage Service)

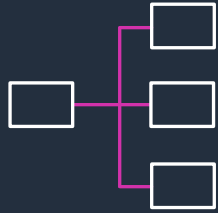
- Almacén de objetos accesible por web (a través de API o HTTPS)
- Altamente durable (diseñado para 99,999999 999%)
- Escalabilidad ilimitada
- Múltiples clases para ajustarse a tu carga de trabajo
- Reglas de ciclo de vida de los datos
- Hosting de sitios estáticos (recomendado: CloudFront + S3)
- Capacidades de seguridad, cumplimiento y auditoría
- Precio de almacenamiento Standard (us-east-1): US\$0,023 por GB



Tu elección de clases de almacenamiento de objetos



Funcionalidades de administración de S3



Organizar

Etiquetas (tags) de S3

Prefijos de S3

Versionado de S3



Monitor

CloudWatch

CloudTrail

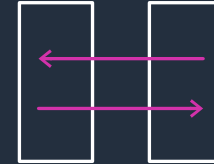
Notificaciones de eventos de S3

Inventario de S3

Notificaciones de restauración de
S3 Glacier

S3 Storage Lens

AWS Config



Replicación

Ciclo de vida de S3

Análisis de clases de
almacenamiento de S3

S3 Intelligent-Tiering

Replicación entre regiones

Control de tiempo de replicación
(RTC)



Edición

Notificaciones de eventos de S3 +
Lambda

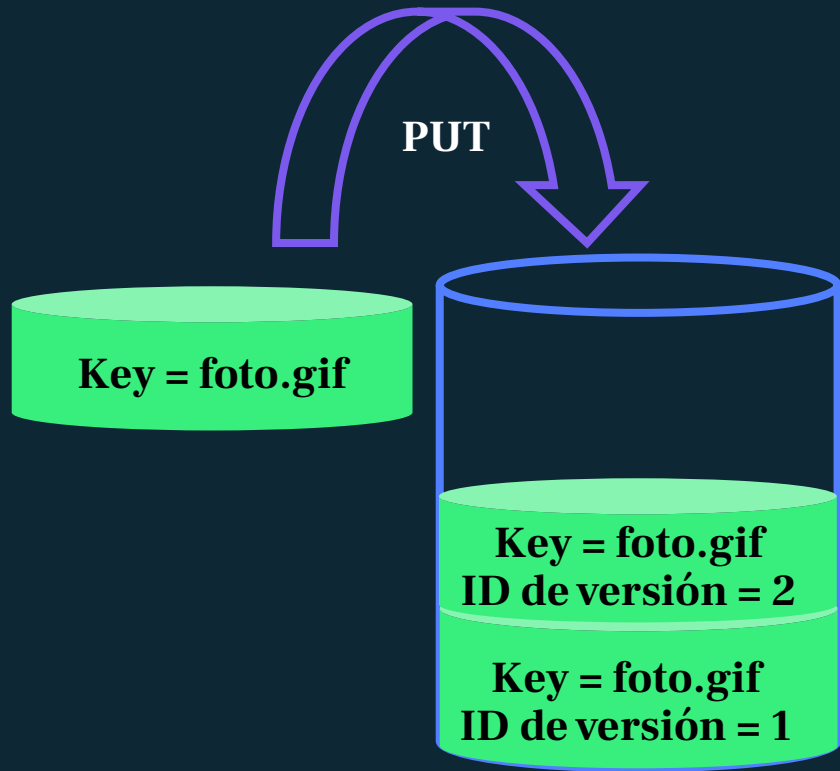
S3 Batch Operations

S3 Object Lock

S3 Object Lambda

Habilita el versionado de buckets de Amazon S3

Usa el versionado para proteger tus datos contra borrados accidentales

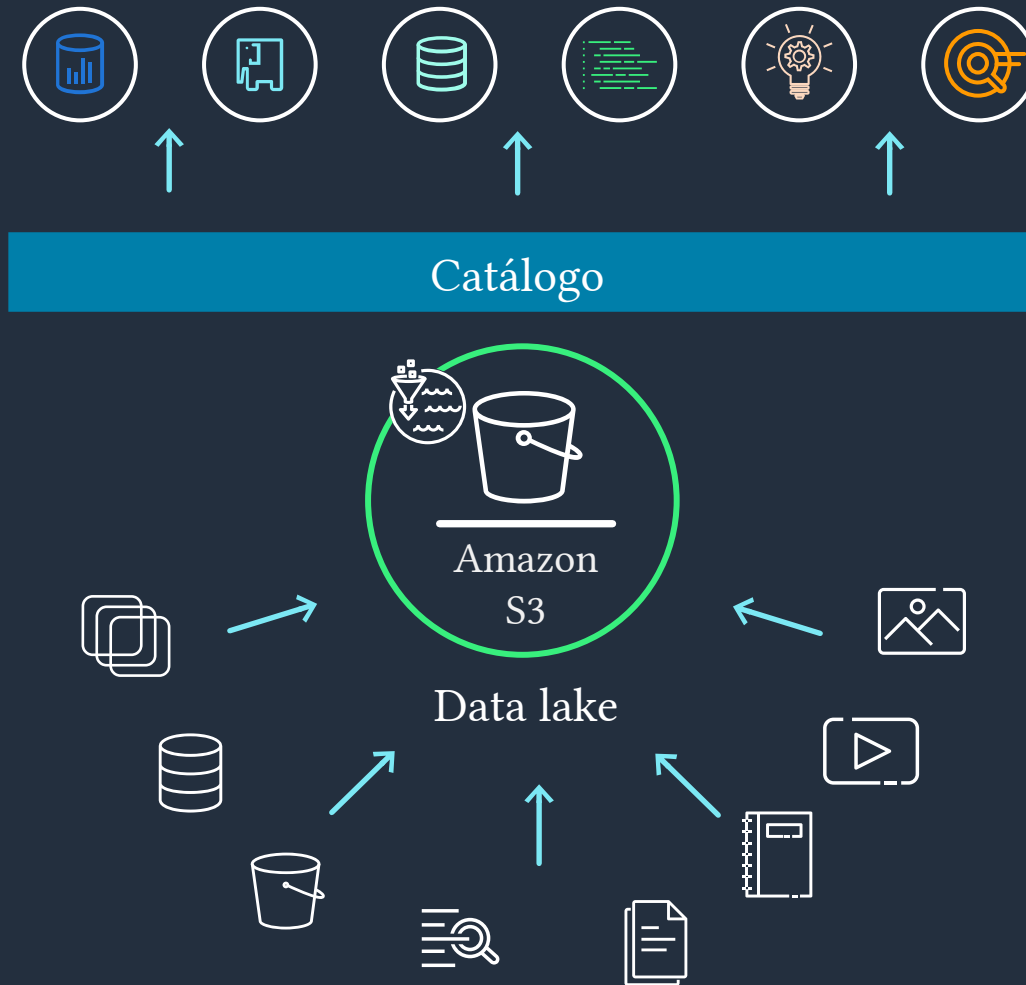


Crea una **nueva versión** con cada carga Las versiones anteriores se conservan, no se sobrescriben

Las **solicitudes de borrado sin ID de versión** quitan el acceso a los objetos, pero conservan los datos

Administra las versiones anteriores con el ciclo de vida
Transiciona o expira los objetos un número de días después de dejar de ser la versión actual

Data lakes escalables (la base es S3)



Guarda todos tus datos en formatos abiertos

Escala el almacenamiento de forma rentable

Desacopla el almacenamiento del cómputo

Elección de motores de analítica y ML

Procesa los datos donde están

Buenas prácticas: Amazon S3

Buenas prácticas

1 Nada público

Activa S3 Block Public Access en cuenta y bucket. Controla el acceso con IAM y políticas de bucket; comparte con URLs prefirmadas o Access Points.

2 Cifrado siempre

Cifrado por defecto en reposo (SSE-S3 o SSE-KMS con llaves propias para datos sensibles) y TLS en tránsito (política que niegue HTTP).

3 Protege los datos

Versionado en buckets críticos, Object Lock (WORM) para retención regulatoria y replicación entre regiones para recuperación ante desastres.

4 Optimiza el costo

Reglas de ciclo de vida para archivar y expirar; Intelligent-Tiering cuando no conozcas el patrón de acceso; Storage Lens para visibilidad.

5 Sitios estáticos

Publica front-ends con CloudFront + S3 (Origin Access Control) manteniendo el bucket privado; no uses el hosting web del bucket.

6 Diseña para escala

Nombres y prefijos pensados para consultas y particiones (p. ej. year=/month=/day=). Usa eventos S3 → Lambda / EventBridge para procesar al llegar.



Base de datos NoSQL: Amazon DynamoDB



¿Diferencias entre DBs relacionales y NO relacionales?

SQL

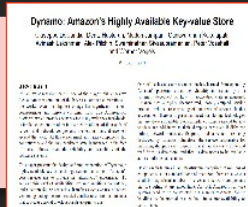
NoSQL

Optimizado para almacenamiento	Optimizado para cómputo
Normalizado / relacional	Desnormalizado / jerárquico
Consultas ad hoc	Vistas materializadas
Escalado vertical	Escalado horizontal
Adecuado para OLTP tradicional	Diseñado para OLTP a gran escala

El recorrido de NoSQL en Amazon

Dec 2004:

Database scalability
challenges



Oct 2007:

Dynamo paper published

Jan 2012:

DynamoDB general
availability



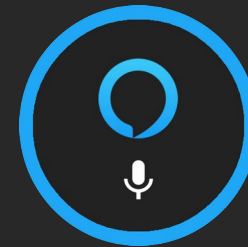
Q3 2016:

DynamoDB leader in
Gartner MQ, Forrester
Wave

Today:

Tier 0 service powering
most of Amazon

amazon



aws

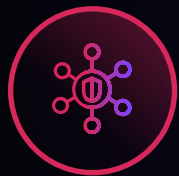
Prime

amazonmusic

¿Por qué Amazon DynamoDB?



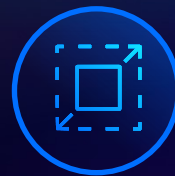
Consistente
Rendimiento
a escala



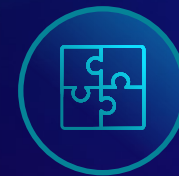
Seguridad y
tolerancia a
fallos



Serverless



Impulsada por
eventos
Arquitectura



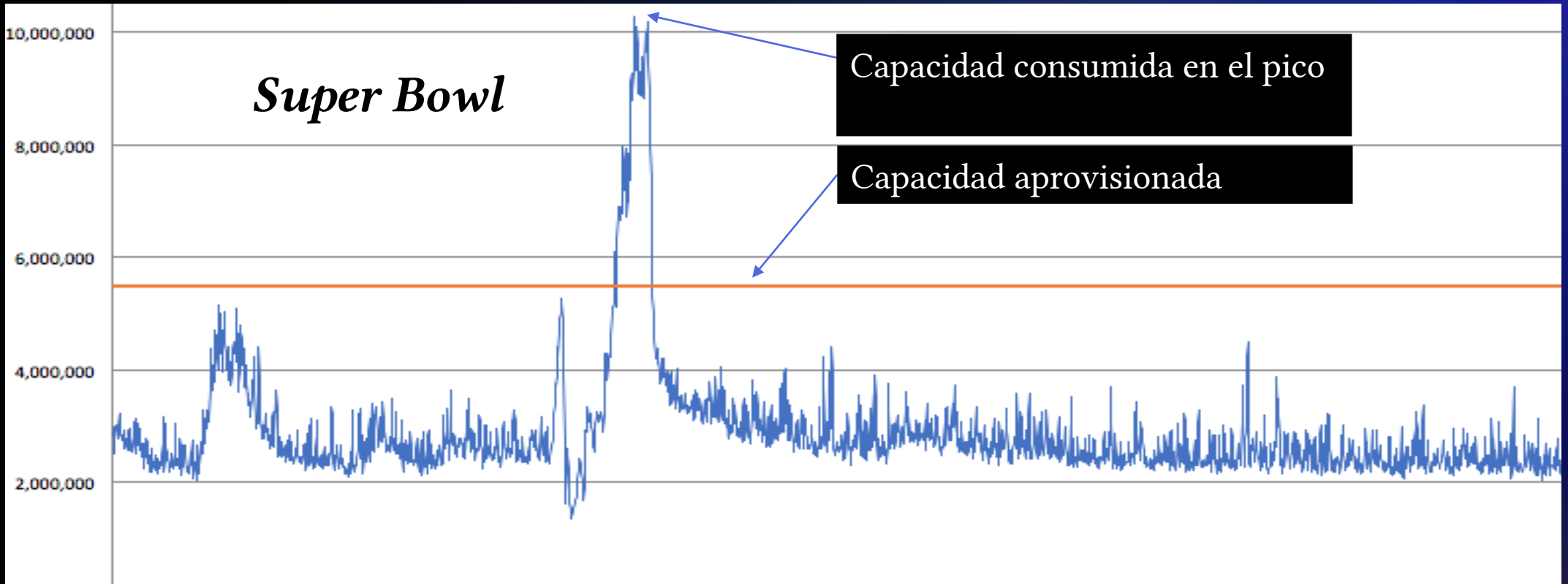
Interoperabilidad
con diversos
servicios



Global

Eventos a escala global: la elasticidad es la nueva normalidad

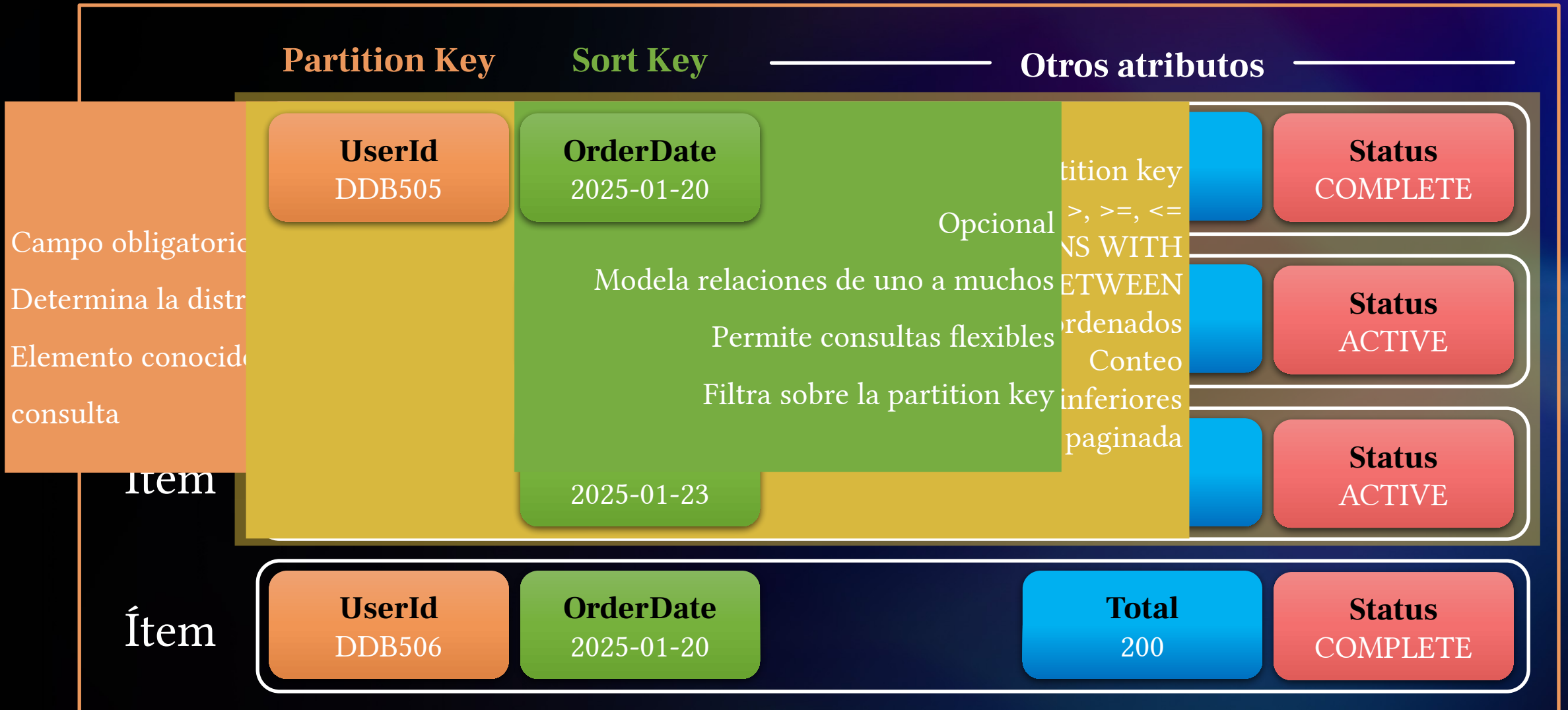
Unidades de capacidad de escritura/s



Una tabla de DynamoDB

	Partition Key	Sort Key	Otros atributos		
Ítem	UserId DDB101	Type METADATA	OrgId A32	Email jdoe@mail.it	ZIP 01 010
Ítem	UserId DDB102	Type METADATA	OrgId A33	Email janej@mail.it	
Ítem	UserId DDB102	Type ORDER#2024		Total 19.99	Status ACTIVE
Ítem	UserId DDB105	Type ADMIN			

Una tabla de DynamoDB



Clave primaria: identifica de forma única un ítem

Clave primaria simple:
clave de partición



SensorId (Clave de partición)	Latitud	Longitud
SensorA	40.712784	-74.005941
SensorB	35.689488	139.691706

Tabla SensorLocation

Clave primaria compuesta: clave
de partición + clave de
ordenamiento



SensorId (Clave de partición)	Tiempo (Clave de ordenamiento)	Valor
SensorA	2018-01-03T10:15:30	30
SensorA	2018-01-04T10:19:30	35
SensorB	2018-03-04T11:21:20	28

Tabla SensorReadings

Funcionalidades de Amazon DynamoDB



Copias de seguridad bajo demanda



Time To Live (TTL)



Recuperación PITR



Rendimiento en memoria

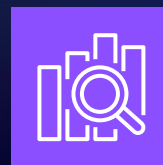


Herramientas de desarrollo compatibles con SQL



Transacciones ACID

Integración con Zero-ETL



Amazon OpenSearch Service



Amazon Redshift



Amazon S3

Servicios de integración adicionales



Compatibilidad con Amazon DynamoDB Streams y Kinesis Data Streams

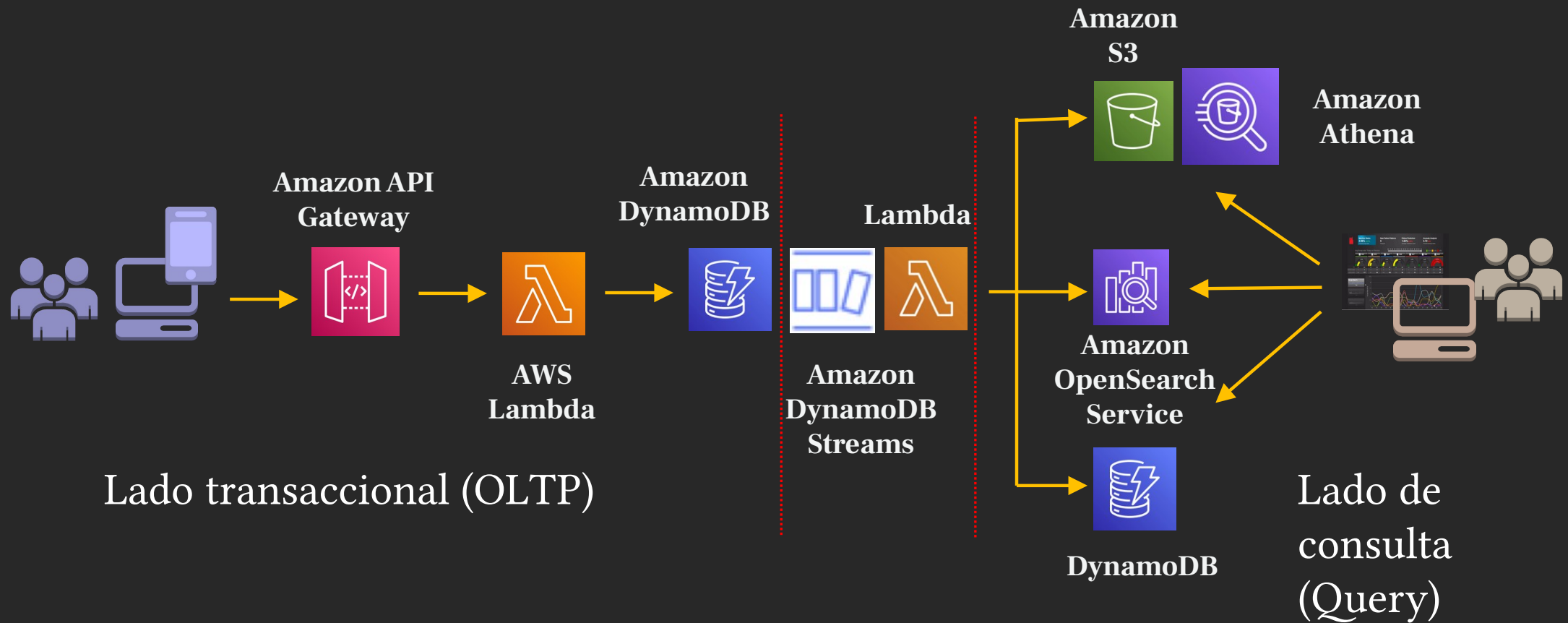


para DynamoDB Amazon CloudWatch Contributor Insights



Completa e incremental Exportación a S3

Ejemplo de aplicación Serverless



Buenas prácticas: Amazon DynamoDB

Buenas prácticas

1 Modela por consultas

Empieza por los patrones de acceso, no por el ERD. Cada consulta debe resolverse con la clave primaria o un índice; evita Scan en producción.

2 Partition key distribuida

Usa claves de alta cardinalidad (customerId, transactionId) para repartir la carga y evitar particiones calientes; añade sufijos si un valor concentra tráfico.

3 Sort keys e índices

Sort keys jerárquicas (2026-09#TX#...) y GSIs dispersos para consultas alternativas. Proyecta solo los atributos que necesitas.

4 Capacidad correcta

On-Demand para cargas variables o nuevas; Provisioned + Auto Scaling cuando el tráfico es estable. Monitorea Throttles y consumo en CloudWatch.

5 Protección y ciclo de vida

Activa PITR y backups, cifrado con KMS y TTL para expirar sesiones/tokens. Usa Streams + Lambda para reaccionar a cambios.

6 Ítems y errores

Ítems pequeños (< 400 KB, grandes en S3 con referencia), BatchGet/Query en vez de Scan, reintentos con backoff exponencial e idempotencia en escrituras.



Base de datos relacional: Amazon Aurora



Amazon Aurora



Amazon Aurora

Rendimiento y disponibilidad inigualables a escala global para PostgreSQL, MySQL y DSQL



Rendimiento y escalabilidad

- 5x el rendimiento de MySQL estándar y 3x el de PostgreSQL estándar
- La base de datos SQL distribuida más rápida con Aurora DSQL
- Escalabilidad prácticamente ilimitada para cualquier demanda
- I/O-Optimized ofrece precios predecibles y mejor relación precio-rendimiento para cargas intensivas en E/S



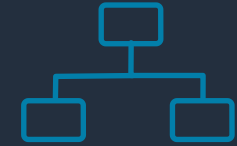
Disponibilidad y durabilidad

- Hasta 99,999% de disponibilidad activo-activo multirregión y hasta 99,99% en una sola región
- Datos durables en 3 AZ (*el cliente paga solo 1 copia*)
- Copias de seguridad automáticas, continuas e incrementales con recuperación a un punto en el tiempo (PITR)
- Recuperación automática ante fallas con Aurora DSQL para máxima resiliencia; failover en < 10 segundos en Aurora PostgreSQL y MySQL
- Almacenamiento tolerante a fallos, autorreparable y con escalado automático



Altamente segura

- Aislamiento de red
- Cifrado en reposo y en tránsito
- Llaves administradas por el cliente mediante la integración con AWS Key Management Service
- Soporta múltiples mecanismos seguros de autenticación y controles de auditoría
- Detección de amenazas basada en ML con Amazon GuardDuty



Totalmente administrada

- Automatiza tareas de administración que consumen tiempo: aprovisionamiento de hardware, configuración, parches y backups
- Serverless basado en API para Aurora DSQL: cero administración de infraestructura, cero tiempo de inactividad y cero ventanas de mantenimiento
- Serverless basado en instancias para Aurora PostgreSQL y MySQL
- Búsqueda vectorial nativa en Aurora PostgreSQL con integración con Amazon Bedrock para RAG automatizado
- Servidores MCP para agentes

Funcionalidades clave de Amazon Aurora



Up to
SLA de 99,99%



Serverless con escalado
instantáneo y granular



Global Database



Cifrado en reposo
y en tránsito



Copias de seguridad automáticas y
continuas



Recuperación a un
punto en el tiempo (PITR)



Datos durables en
3 AZ (Multi-AZ)



Compatibilidad total con
MySQL y PostgreSQL



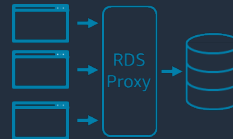
Clonación de
bases de datos



Escala horizontal hasta
15 réplicas de lectura



Almacenamiento tolerante a fallos,
autorreparable y con escalado
automático



RDS Proxy

Ejemplos de integración de Aurora con otros servicios de AWS



Amazon EKS



Amazon Lambda



AWS CloudTrail



Amazon S3



Amazon IAM



Amazon KMS



Amazon
Sagemaker



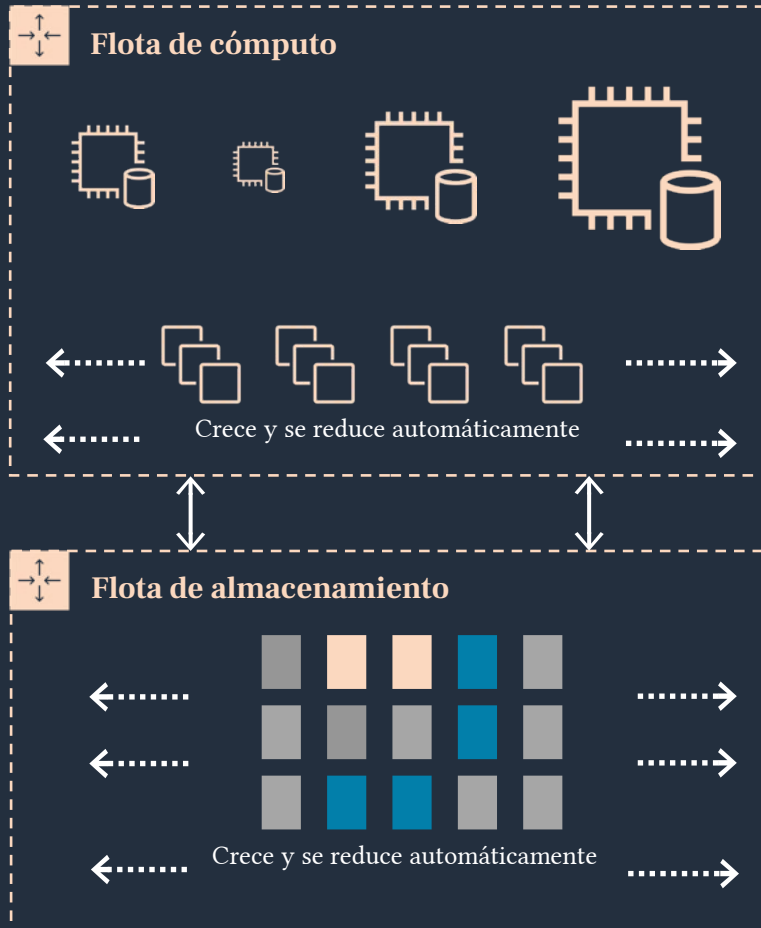
Amazon
DevOps Guru



Amazon
CloudWatch

Amazon Aurora Serverless v2: escalado instantáneo en incrementos finos, reduciendo costos

Aplicaciones



- Escala en incrementos finos para entregar justo la capacidad de base de datos necesaria según la demanda de los eventos de tu aplicación
- Escala al instante, en una fracción de segundo, incluso para las aplicaciones más exigentes
- Hasta 90% de ahorro de costos frente a aprovisionar para el pico de carga
- Toda la funcionalidad de Aurora, incluyendo Parallel Query, Global Database, réplicas de lectura y soporte Multi-AZ

¿Cuándo usar Aurora y cuándo DynamoDB?

Criterio	Amazon Aurora (relacional)	Amazon DynamoDB (NoSQL)
Modelo de datos	Tablas relacionadas, esquema fijo, SQL completo (joins, agregaciones, vistas).	Clave-valor / documentos, esquema flexible; se modela por patrones de acceso.
Escalado	Vertical + hasta 15 réplicas de lectura; Serverless v2 escala cómputo automáticamente.	Horizontal y prácticamente ilimitado; particiona los datos de forma automática.
Latencia y rendimiento	Milisegundos; excelente para consultas complejas y reportes transaccionales.	Milisegundos de un dígito constantes a cualquier escala; microsegundos con DAX.
Transacciones	ACID completas, multi-tabla, integridad referencial nativa.	ACID por ítem y transacciones de hasta 100 ítems; sin claves foráneas.
Casos típicos	Core bancario, ledger contable, ERP/CRM, reportes con SQL, migración desde MySQL/PostgreSQL.	Carritos, sesiones, perfiles, historial de transacciones, IoT, catálogos, APIs de alta concurrencia.
Operación y costo	Instancias o ACUs + almacenamiento + I/O; requiere afinar índices y conexiones (RDS Proxy).	Pago por solicitud (On-Demand) o capacidad; sin servidores ni parches; costo predecible por acceso.



Buenas prácticas: Amazon Aurora

Buenas prácticas

1 Alta disponibilidad

Clúster Multi-AZ con al menos una réplica de lectura. Usa los endpoints de writer y reader; el failover automático toma segundos.

2 Serverless v2 bien dimensionado

Define ACU mínimo/máximo según el ambiente (dev bajo, prod con margen). Revisa las métricas ServerlessDatabaseCapacity antes de fijar límites.

3 Conexiones controladas

Con Lambda y microservicios usa RDS Proxy para pooling y evitar agotar conexiones; configura timeouts y reintentos en el cliente.

4 Respaldo y pruebas

Backups automáticos con PITR, snapshot antes de cada cambio de esquema y clonación (copy-on-write) para pruebas sin tocar producción.

5 Seguridad de red y datos

Subredes privadas, security groups solo desde la aplicación (nunca 0.0.0.0/0), cifrado con KMS, credenciales en Secrets Manager con rotación o IAM auth.

6 Observabilidad

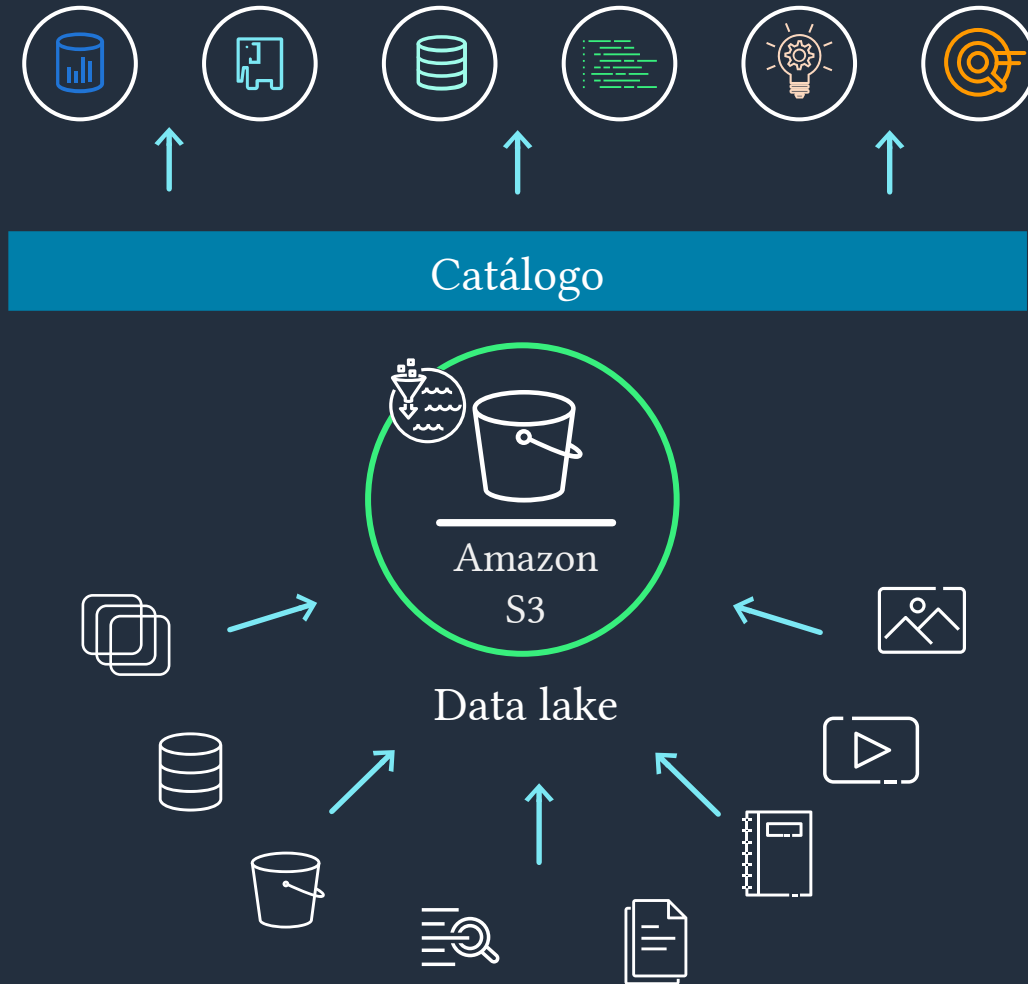
Performance Insights para consultas lentas, CloudWatch y DevOps Guru para anomalías. Revisa índices, VACUUM/ANALYZE y planes de ejecución.





Analítica serverless: Amazon Athena

Data lakes escalables



Guarda todos tus datos en formatos abiertos

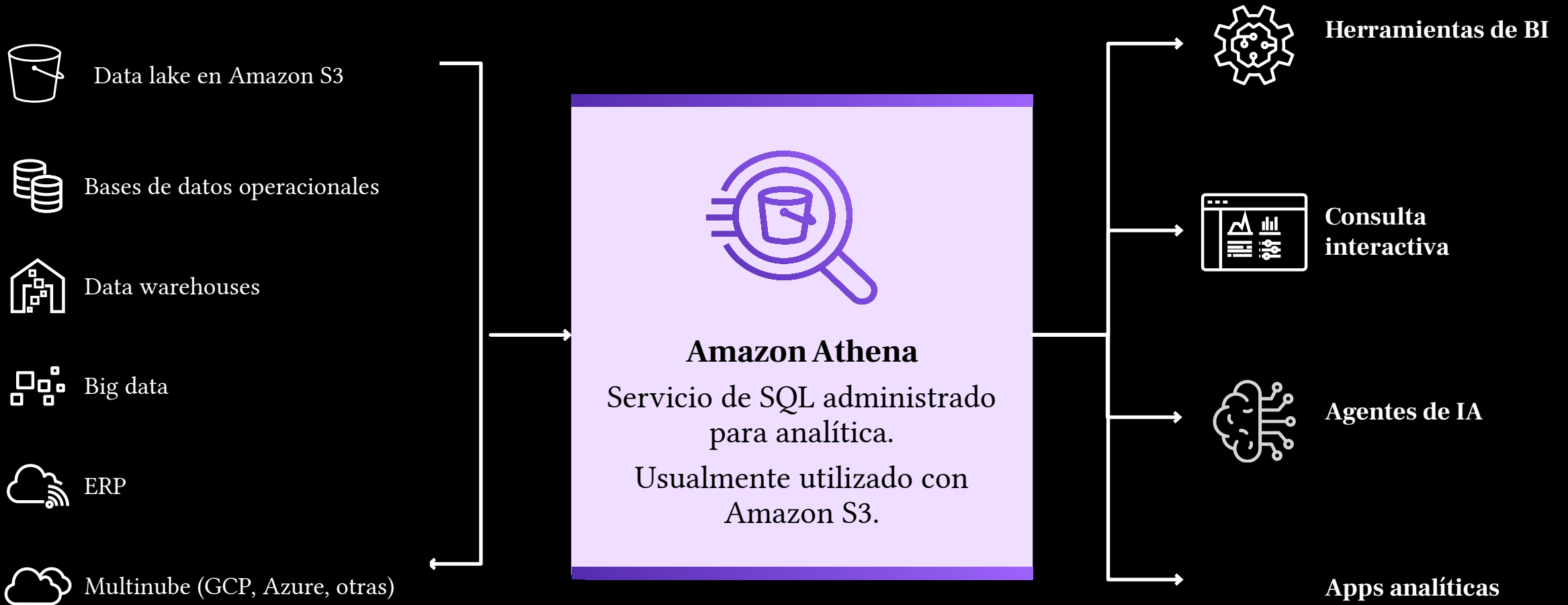
Escala el almacenamiento de forma rentable

Desacopla el almacenamiento del cómputo

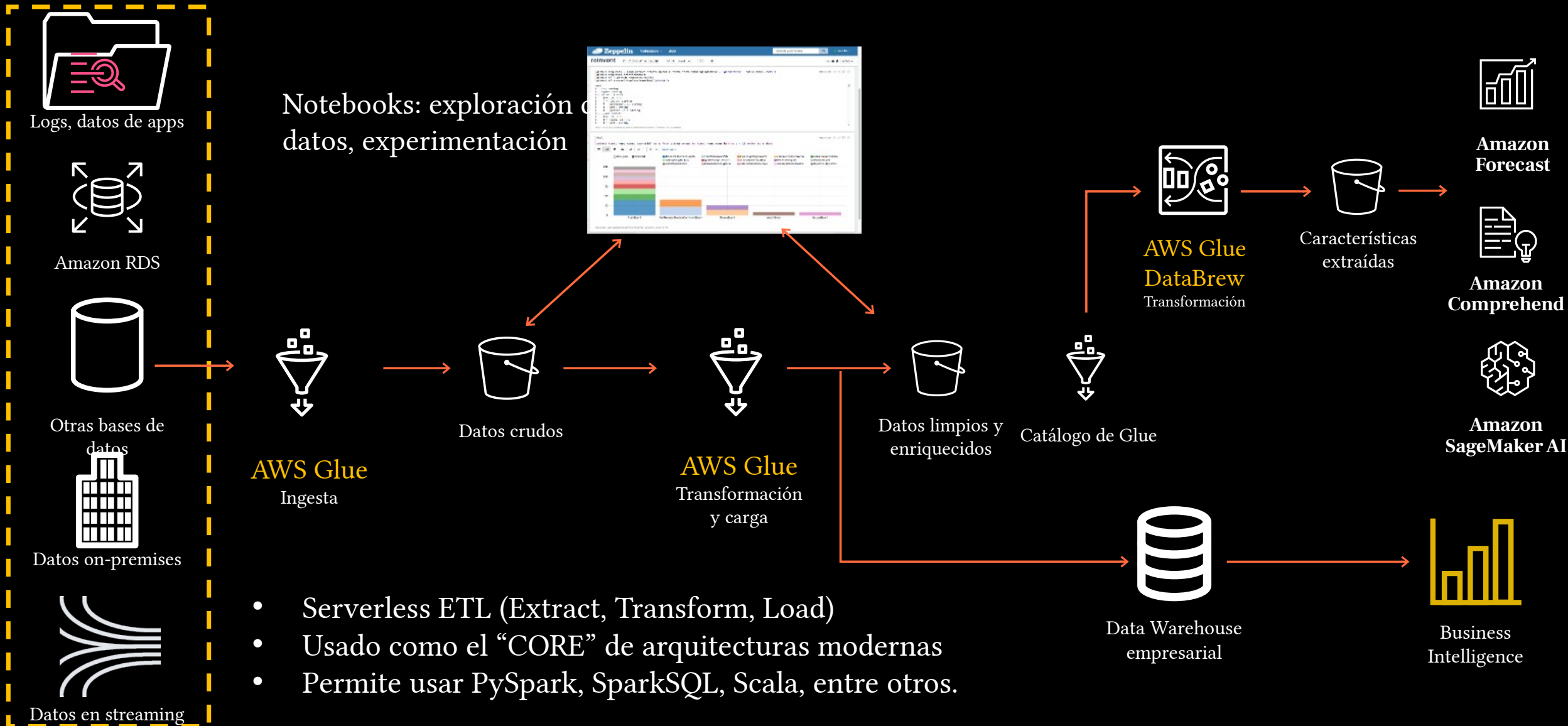
Elección de motores de analítica y ML

Procesa los datos donde están

Amazon Athena – Serverless SQL (bajo demanda)



AWS Glue – El Corazón de los ETLs en la Nube





Amazon Athena

Simple, flexible y fácil de usar



Fácil de usar

Serverless: sin infraestructura que configurar ni administrar

Funciona con las herramientas que usas hoy



Optimizado para interactividad

Tiempo de inicio casi instantáneo

Resultados rápidos: la mayoría de consultas termina en segundos



Funciona con tus datos

Soporta formatos de datos abiertos populares y fuentes federadas

Gobierno de datos granular

Impulsa agentes de IA con acceso federado a datos y arranque de consultas en menos de un segundo con Athena para Apache Spark



Precios transparentes y predecibles

Precios flexibles de pago por uso

Sin cargo adicional por el cómputo subyacente



Precios de Athena

- **Pago por consulta**
\$5/TB

- La forma más fácil de empezar
- Paga solo por las consultas ejecutadas
- Ahorra hasta 90% por consulta con compresión, particionamiento y formatos columnares

- **Pago por cómputo**

\$0,30/DPU-hora *

- Reserva capacidad para tus cargas de trabajo por anticipado
- Paga solo por la capacidad aprovisionada y el tiempo activo
- Unidades de procesamiento de datos (DPU): 4 vCPU y 16 GB de RAM



Puedes usar ambos modelos al mismo tiempo en la misma cuenta
Por ejemplo, facturación por consulta en desarrollo y por cómputo en producción

* Precios de US-EAST-1. Consulta <https://aws.amazon.com/athena/pricing/> para otras regiones



Buenas prácticas: Amazon Athena

Buenas prácticas

1 Formatos columnares

Guarda en Parquet u ORC con compresión (Snappy/ZSTD). Menos bytes escaneados = consultas más rápidas y más baratas (hasta 90%).

2 Particiona y agrupa

Particiona por fecha u otra dimensión de filtro frecuente (year=/month=/day=) y usa partition projection. Evita miles de archivos pequeños: apunta a 128 MB-1 GB.

3 Catálogo central

AWS Glue Data Catalog como única fuente de esquemas; crawlers o definiciones en IaC; documenta tablas y controla acceso con Lake Formation / IAM.

4 Workgroups y límites

Separa equipos y ambientes con workgroups; fija límites de datos escaneados por consulta, etiqueta para asignar costos y guarda métricas en CloudWatch.

5 Escribe SQL eficiente

Selecciona solo las columnas necesarias, filtra por particiones, usa LIMIT al explorar y CTAS/INSERT INTO para materializar resultados frecuentes.

6 Resultados y seguridad

Bucket de resultados cifrado, con ciclo de vida corto y sin acceso público. Considera Iceberg cuando necesites actualizaciones, borrados o viaje en el tiempo.





Business Intelligence: Amazon QuickSight

Amazon QuickSight

Business Intelligence (BI) a Gran Escala.

Tip: palabra clave es “Dashboards o BI”



Servicio de BI para entrega de DASHBOARDS, reportería y analítica embebida.

Rendimiento a gran escala, distribuido.

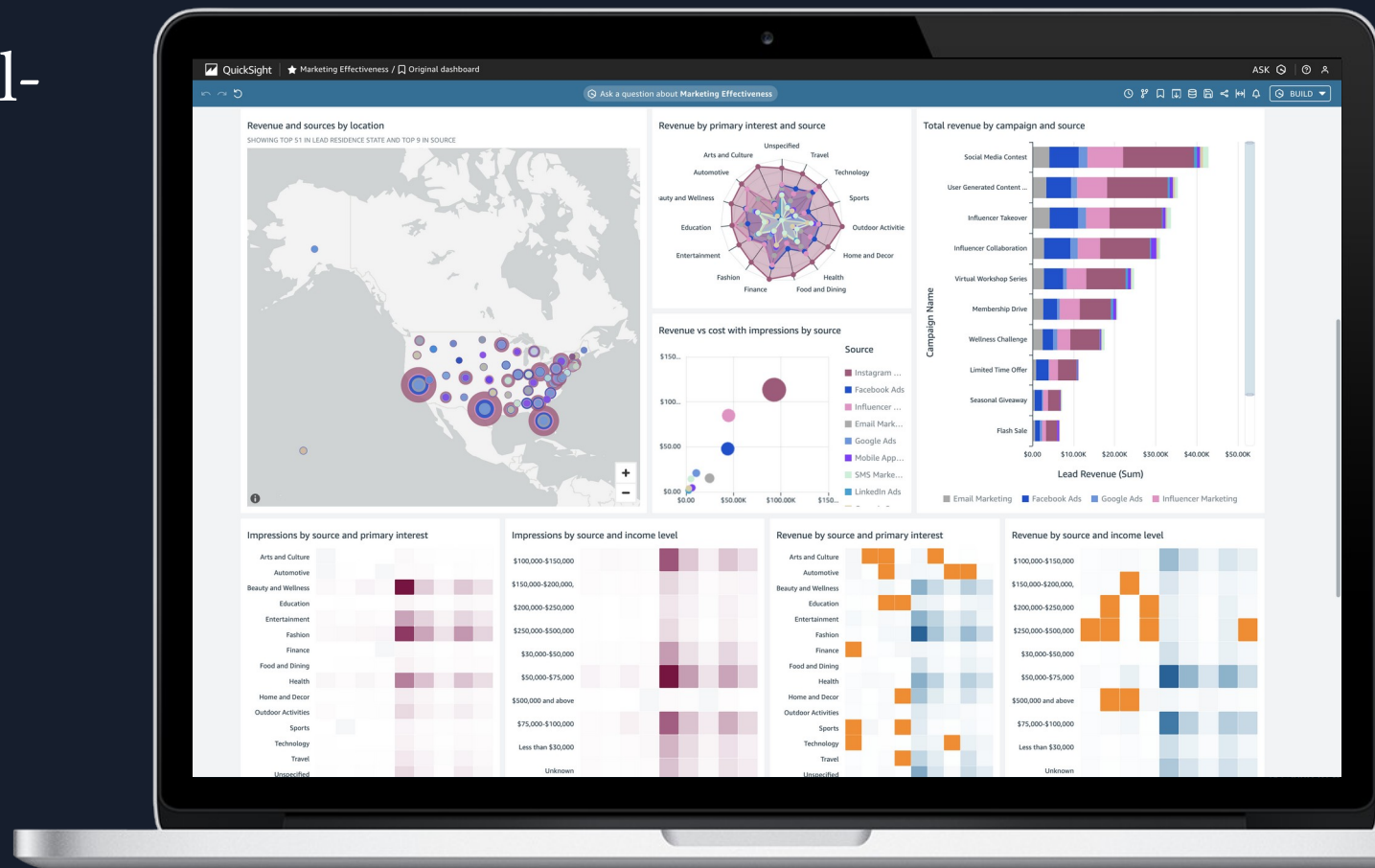
Diversos modelos de costo, según uso y por licencias

Controles y seguridad avanzada



Crea dashboards Interactivos

- Interactividad rica: filtros, drill-down y zoom
- Navegación ultrarrápida
- Accesible desde cualquier dispositivo
- Refresco de datos
- Publicación en un solo paso



Amazon QuickSight

CLOUD-NATIVE BI SOLUTION
FOR ILLUMINATING
ORGANIZATIONAL INSIGHTS



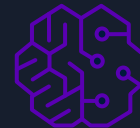
Auto-scaling and serverless



Internal and/or external users



Deeply integrated with AWS services



Augmented insights on-demand

Amazon Quick Suite – (Evolución QuickSight...)



Spaces

- Organiza archivos, dashboards y fuentes de datos
- Espacios colaborativos para tus proyectos



Chat Agents

- Asistentes de IA personalizados con el conocimiento de tu negocio
- Comparte y colabora entre equipos



Research

- Análisis profundo y reportes completos
- Documentación profesional y exportable



Quick Sight

- Business Intelligence y visualización de datos
- Dashboards interactivos y analítica



Flows

- Automatiza tareas repetitivas con pasos predefinidos
- Automatización de flujos sin código



Automate

- Automatización de flujos complejos de varios pasos
- Transforma procesos de negocio completos

Gobierno y seguridad de datos

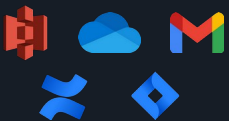
Controles de acceso

Guardrails

IA responsable

Cumplimiento regulatorio

Datos de la empresa: Spaces y datasets



40+ conectores de datos



Archivos cargados por usuarios



Datos de QuickSight

Conocimiento del mundo



Modelos de Bedrock



Búsqueda web

Acciones



Acciones en apps de terceros



Buenas prácticas: Amazon QuickSight

Buenas prácticas

1 SPICE primero

Importa a SPICE para dashboards rápidos y para no golpear Athena/Aurora en cada visualización; usa consulta directa solo cuando necesites datos casi en tiempo real.

2 Datasets gobernados

Pocos datasets compartidos y bien nombrados (una fuente de verdad) en lugar de uno por dashboard. Documenta campos y cálculos.

3 Seguridad por filas

Row-level security con grupos (país, unidad, rol) para que cada usuario vea solo sus datos. Integra con IAM Identity Center.

4 Dashboards enfocados

KPIs arriba, pocos visuales por hoja, filtros y parámetros. Cada dashboard responde una pregunta de negocio concreta.

5 Refrescos y alertas

Programa refrescos de SPICE alineados con la llegada de datos; configura alertas de umbral y anomalías; habilita Q para autoservicio.

6 Costo por rol

Asigna Reader a la mayoría y Author solo a quien construye; revisa usuarios inactivos. Usa embebido para exponer datos en tus propias aplicaciones.





Infraestructura como Código (IaC): Terraform

Infraestructura como Código (IaC)



AWS CDK



AWS CloudFormation



AWSTemplateFormatVersion: '2010-09-09'

Description: Creates an EC2 Instance

Resources:

MyEC2Instance:

Type: 'AWS::EC2::Instance'

Properties:

ImageId: 'ami-0ff8a91507f77f867'

InstanceType: t2.micro

¿Qué logra?

- Escribir código para crear, configurar y desplegar componentes de infraestructura
- La infraestructura incluye: redes, cómputo, bases de datos, seguridad, herramientas de gestión, etc.

¿Por qué?

- Hace que los cambios de infraestructura sean repetibles y predecibles
- [Documenta tu infraestructura](#)
- Automatiza el proceso de aprovisionamiento
- Elimina la deriva de configuración (drift) mediante automatización



¿Qué es Terraform?

Herramienta de IaC declarativa y multi-proveedor (HashiCorp / OpenTofu)

Declarativo

Describes el estado deseado en HCL; Terraform calcula qué crear, cambiar o destruir.

Providers

Plugins para hablar con APIs: AWS, Azure, GCP, Kubernetes, GitHub, Datadog... Una sola herramienta, muchas plataformas.

Estado (state)

Archivo que relaciona tu código con los recursos reales. Se guarda remoto (S3) con bloqueo para trabajar en equipo.

Módulos

Bloques reutilizables y versionados (p. ej. 'lambda-api': función + rol + logs + API Gateway).

Flujo de trabajo

1

Write

Escribe la infraestructura en archivos .tf (HCL) y súbela a Git.

2

terraform init

Descarga providers y módulos; configura el backend remoto del estado.

3

terraform plan

Muestra el diff: qué se crea (+), cambia (~) o destruye (-). Se revisa en el Pull Request.

4

terraform apply

Ejecuta el plan aprobado desde el pipeline de CI/CD y actualiza el estado.

5

terraform destroy

Elimina lo creado (ambientes efímeros, laboratorios, pruebas).



Terraform en la práctica: ejemplo mínimo

Estado remoto + tabla DynamoDB + bucket S3 privado + función Lambda

```
terraform {  
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }  
  backend "s3" {  
    # estado remoto, cifrado y con bloqueo  
    bucket = "mi-empresa-terraform-state" key = "basics/dev.tfstate"  
    region = "us-east-1" encrypt = true use_lockfile = true  
  }  
}  
  
provider "aws" {  
  region = "us-east-1"  
  default_tags { tags = { project = "aws-basics", env = "dev", owner = "mi-empresa" } }  
}  
  
resource "aws_dynamodb_table" "transactions" {  
  name = "mi-empresa-transactions" billing_mode = "PAY_PER_REQUEST"  
  hash_key = "customerId" range_key = "transactionDate"  
  attribute { name = "customerId" type = "S" }  
  attribute { name = "transactionDate" type = "S" }  
  point_in_time_recovery { enabled = true }  
}  
  
resource "aws_s3_bucket" "files" { bucket = "mi-empresa-basics-files-dev" }  
resource "aws_s3_bucket_public_access_block" "files" { # bucket privado, siempre  
  bucket = aws_s3_bucket.files.id  
  block_public_acls = true block_public_policy = true  
  ignore_public_acls = true restrict_public_buckets = true  
}  
  
resource "aws_lambda_function" "api" {  
  function_name = "mi-empresa-basics-api" runtime = "python3.13"  
  handler = "app.handler" role = aws_iam_role.lambda.arn  
  filename = "build/app.zip" timeout = 10  
  environment { variables = { TABLE_NAME = aws_dynamodb_table.transactions.name } }  
}
```

terraform / backend

Fija versiones del provider y guarda el estado en S3, cifrado y con bloqueo (use_lockfile).

provider 'aws'

Región y default_tags: todos los recursos nacen etiquetados (owner, env, project).

resource

Cada bloque = un recurso real. Terraform resuelve las dependencias por las referencias (aws_s3_bucket.files.id).

Seguridad incluida

Bucket privado (public_access_block), tabla con PITR y rol IAM propio para la Lambda.

Ciclo

terraform init → plan → apply. El plan muestra exactamente qué cambia antes de tocar la cuenta.



Terraform vs. CloudFormation vs. AWS CDK

Aspecto	Terraform / OpenTofu	AWS CloudFormation	AWS CDK
Lenguaje	HCL (declarativo) o CDKTF.	YAML / JSON (declarativo).	TypeScript, Python, Java, C#, Go; genera CloudFormation.
Alcance	Multi-nube y SaaS (AWS, Azure, GCP, GitHub, Datadog...).	Solo AWS; nativo y sin costo adicional.	Solo AWS; construcciones de alto nivel con buenas prácticas incluidas.
Estado	Archivo de estado gestionado por ti (S3 + bloqueo).	Lo gestiona el servicio (stacks); detección de drift integrada.	Igual que CloudFormation (stacks).
Ecosistema	Registry con miles de módulos y providers; gran comunidad.	Integrado con Service Catalog, StackSets, Control Tower.	Constructs Hub, pruebas unitarias, abstracciones reutilizables.
Ideal para	Equipos multi-nube o con Terraform ya adoptado; plataformas de infraestructura.	Gobierno centralizado 100% AWS, landing zones, cumplimiento.	Desarrolladores que prefieren código y aplicaciones serverless complejas.



Resumen: lo que construimos hoy

AWS y la nube

Recursos bajo demanda, pago por uso, regiones y AZs; responsabilidad compartida.

AWS Lambda

Ejecuta código por eventos sin servidores; escala automático; paga por milisegundo.

Amazon S3

Objetos con 11 nueves de durabilidad; clases y ciclo de vida; nunca público.

Amazon DynamoDB

NoSQL serverless de milisegundos; modela por patrones de acceso; PK + SK + GSI.

Amazon Aurora

Relacional MySQL/PostgreSQL, 6 copias en 3 AZ; Serverless v2 escala fino.

Amazon Athena

SQL serverless sobre S3 con Glue Data Catalog; paga por datos escaneados.

Amazon QuickSight

BI serverless: dashboards, SPICE, row-level security y Amazon Q.

Terraform (IaC)

Infraestructura declarativa y versionada: init → plan → apply desde CI/CD.



DEMOS EN VIVO



¿Preguntas?

